A BM GUIDE

Characters and Game Models: sprites, 2D pre-renders and 3D

How to make the models of playable characters, NPCs and game elements: sprites, pre-rendered 2D and 3D.

Contents

What this manual covers . . . 5

The tools and what each one makes . . . 5

Good to know . . . 5

Caveats . . . 5

Choosing a technique . . . 7

What it does . . . 7

Why it matters . . . 7

Good to know . . . 7

Sprites by hand . . . 8

What it does . . . 8

The calls a game uses . . . 8

Good to know . . . 8

Caveats . . . 8

Side-scroller characters . . . 9

What it does . . . 9

What to draw . . . 9

How a game draws it . . . 9

Collision and movement . . . 9

Hitboxes of the frames . . . 9

Top-down characters, 4 directions . . . 10

What it does . . . 10

What to draw . . . 10

Good to know . . . 10

Top-down characters, 8 directions . . . 11

What it does . . . 11

Mirroring is the saving . . . 11

One module for all the animations . . . 11

From the keys to the direction . . . 12

How to make the 8 directions . . . 12

Optimising sprites . . . 13
Mirroring . . . 13
Reusing parts of sprites . . . 13
Palettes . . . 13
Darkness and glow with the same palette . . . 13
Sprite stacking . . . 14

Pre-rendered 2D . . . 16
What it does . . . 16
How the games of the repository do it . . . 16
The same thing on the console: bm Animator F4 . . . 16
The hitboxes from the bones . . . 17
Touch up with bm Pixel . . . 17
Good to know . . . 17
What F4 does not do yet . . . 17
Try it, step by step . . . 17

Meshes . . . 19
What it does . . . 19
From an image to a mesh . . . 19
From a mesh to an image . . . 19
From a mesh to a model and back, with bm Mesh . . . 20
Editing vertices and faces . . . 20
Where the game's meshes come from . . . 20
Caveats . . . 20

3D models . . . 21
What it does . . . 21
bm Studio: the model . . . 21
bm Animator: the skeleton and the animations . . . 21
Name the bones with the game in mind . . . 21
Models outside the console . . . 21
Caveats . . . 22

Drawing and animating in the game . . . 23

The model and its animation . . . 23
Light, shadow, shading . . . 23
The flags of `draw3d` . . . 24
Materials in a face's colour . . . 24
Shaders . . . 24
Code to copy . . . 24

Tutorials . . . 26
A side-scroller hero . . . 26
A top-down hero with 4 directions . . . 26
A 3D character baked into 8 directions . . . 26

Tips and limits . . . 27
Limits . . . 27
Tips . . . 27
When something goes wrong . . . 27

Open points . . . 28

README

What this is. The guide to making the *models* of a game: the hero, the enemies, the townsfolk, the props. It joins what the editors of the suite, the API and the repository's games already do.

How to use it. Read the first two chapters to choose a technique, then go to the chapter of that technique. Anything in a bordered box with a shadow is meant to be copied as it is.

What is missing. What does not exist yet, or exists in part, is marked **[IN DEVELOPMENT]** with one line of what is missing. What is planned for the first beta revision says "beta rev 1". Nothing is invented.

What this manual covers

A character in bm is one of three things: a **sprite** in the sprite sheet, a set of **pre-rendered sprites** (a 3D model drawn into the sheet beforehand) or a **3D model** drawn live. This manual is about making them. It is not about the game's code around them, which is in the API manuals.

The tools and what each one makes

Tool	What it makes	Chapter
bm Pixel	sprites, tiles, palettes; touches up baked sprites	Sprites by hand
bm Animator, page F4	pre-rendered 2D sprites from a 3D model, with hitboxes from the bones	Pre-rendered 2D
bm Studio	3D models from blocks and tiles; a model from a picture	3D models
bm Animator, pages F2 and F3	the skeleton and the animations of a model	3D models
bm Mesh	the vertices and faces of a model; meshes built by a game's code	Meshes
`scripts/bmres.py` , `tools/`	extract, add and convert resources; models from pictures, on the PC	Meshes, Pre-rendered 2D
bm-forge	models, skeletons and previews from a command line, on the PC	3D models

Good to know

- Pixel, Studio, Animator and Mesh work on the same file: a **project** (`.bme`). A game (`.bm` , `.b16`) is read only, and the first save makes an editable `.BME` copy.
- The sprite sheet is shared: bm Pixel draws it, bm Studio uses pieces of it as textures, bm Animator puts baked sprites into it.
- The pad and keys of each editor are in its own manual (`editor-pixel` , `editor-studio` , `editor-animator` , `editor-mesh`). Here only what a character needs.

Caveats

- The manual of bm Animator says that the kernel of the RGB30 does not have it, so baking sprites on the RGB30 console is **[IN DEVELOPMENT]**: not available there today.

Choosing a technique

What it does

The same hero can be made three ways. Pick by the look you want and by how many frames you can afford.

Technique	Look	Cost	Good for
Hand-drawn sprites	pixel art, your own hand	every frame is drawn by you	platformers, small top-down games, props
Pre-rendered 2D	pixel art from a 3D model: the same figure from many sides, always in proportion	a model, a skeleton, then one bake	fighters, 3/4 view games with 8 directions
3D model	real 3D, turns and moves freely	about 1200 triangles a scene at 60 fps	dioramas, flight, free cameras

Why it matters

The notes of the repository are clear on the limit: drawing every direction of every action by hand takes months and is hard to keep consistent. *Yharnam* has over 5,000 frames for the hunter and the creatures. That is why pre-rendering exists.

Good to know

- *Titan Clash* (`carts/titan`) and *Yharnam* (`carts/yharnam`) use pre-rendered sprites.
- A 3D model can feed the other two: the same model is a 3D character and, baked, a 2D one.

Sprites by hand

What it does

A sprite is a piece of the **sprite sheet**, the one picture a game reads. The sheet is divided in cells of 8x8; a sprite is one cell or several. bm Pixel draws it enlarged, pixel by pixel, at the size 8, 16, 32, 64 or 128 pixels (**z** changes the size).

The calls a game uses

Call	What it does
`spr(n, x, y, [w, h, flip_x, flip_y])`	draws cell `n`, `w` by `h` cells, mirrored too
`sspr(sx, sy, sw, sh, dx, dy, [flip_x, flip_y, zoom])`	copies any rectangle of the sheet, mirrored too, bigger or smaller with `zoom`
`zspr(name, x, y, [frame, flip_x, flip_y, zoom])`	draws a named zone; without `frame` it animates by itself
`zone(name)`	`x, y, w, h, frames, fps, slices` of the zone
`zboxes(name, [frame, kind])`	the hitboxes and hurtboxes of the zone's frames

A zone is a name for a rectangle of the sheet, its frames (boxes of the same size side by side on a row) and its speed. See the manual of bm Pixel for the frames and fps shown in the editor.

Good to know

- Colour 0 is transparent. In a PNG sheet, alpha under 128 is transparent.
- The sheet is up to 4096 x 4096 pixels.
- Draw the frames of an animation one after the other, left to right, on the same row: that is how a zone finds them.

Caveats

- **[IN DEVELOPMENT]** Naming a zone in bm Pixel. Today zones are written in a text file (`name x y width height [frames [fps]]`) and given to `mkbm.py --sprites` . Planned: beta rev 1.
- The frames and the fps you set in bm Pixel are remembered for the file but are not written into the cartridge.

COPY AND PASTE

```
hero_still   0  32 16 16
hero_run     16 32 16 16 4 10
```

Side-scroller characters

What it does

A platformer hero is seen from the side. You draw it once, facing right, and the game mirrors it when it faces left.

What to draw

- One zone per action, facing right: `hero_still` , `hero_run` , `hero_jump` . Each one a row of equal frames.
- The speed of the cycle is in the zone (`fps`).
- bm Pixel offers the sizes 8, 16, 32, 64 and 128 pixels; a size that is a multiple of the map's tiles keeps the feet on the ground cells.

How a game draws it

COPY AND PASTE

```
local facing_left = hero.vx < 0
zspr("hero_run", hero.x, hero.y, nil, facing_left)
```

The same call with a frame number (`zspr("hero_run", x, y, 2, facing_left)`) draws that frame only: use it for a jump, where the frame follows the speed and not the clock.

Collision and movement

- Solid ground, platforms, ladders, water and spikes are **tile flags** (0 solid, 1 platform, 2 ladder, 3 water, 4 hurts), read with `fget(n, f)` and `mflags(x, y, [w, h, layer])` .
- bmlib moves a body for you: `lib.move(b, dx, dy)` stops it against solid tiles, `lib.step(b)` adds gravity.
- The bm SDK has a template, **Platform 2D**: run and jump on the map, with the camera following.

Hitboxes of the frames

For a fighter, the BOXES section gives each frame its `hurt` , `hit` and `body` rectangles, read with `zboxes` . bmlib's `H:zone(...)` puts them in the world with the same flip as the sprite.

BEWARE

[IN DEVELOPMENT] Drawing boxes in bm Pixel. bm Pixel shows the boxes (**j**) but does not draw, move or resize them: they are written in the zones text file. Planned: beta rev 1.

Top-down characters, 4 directions

What it does

Seen from above, the character faces down, up, left or right. Four directions need only three drawings: down, up and one side, because the other side is the first one mirrored.

What to draw

A game does this with sprites of 2 by 2 cells and two walking frames for each view:

- facing down: the first sprite of the group, then the second walking frame;
- facing up: the sprite 4 cells after;
- facing a side: the sprite 8 cells after, mirrored when it walks to the left.

COPY AND PASTE

```
local s, flip
if math.abs(p.fx) > 0.3 then s, flip = BASE + 8 + fr * 2, p.fx < 0
elseif p.fy < 0 then s, flip = BASE + 4 + fr * 2, false
else s, flip = BASE + fr * 2, false end
spr(s, math.floor(p.x) - 8, math.floor(p.y) - 14, 2, 2, flip)
```

Good to know

- The sprite is drawn with its feet near the point `p.x, p.y`, so the order of drawing by `y` works: what is lower on the screen is in front.
- The SDK has a template, **Top-down 2D**: walk among walls and pick up coins.
- The calls are the same as the side-scroller ones (`spr` , `sspr` , `zspr` with `flip_x`).

Top-down characters, 8 directions

What it does

Eight directions (S, SE, E, NE, N, NW, W, SW) are what a 3/4 view game like *Yharnam* uses: the character walks and attacks in any of them.

Mirroring is the saving

Five directions are drawn (S, SE, E, NE, N). The other three (SW, W, NW) are the first ones mirrored. *Yharnam* does it for the creatures, which are drawn in 5 directions; the hunter has the 8 directions drawn in the sheet.

COPY AND PASTE

```
-- the frame on screen: the drawn direction, mirrored for SW W NW
function ANIM.frame(o, set, dir)
  local d = set[o.anim].d
  if #d == 8 then return d[dir + 1][o.f], false end   -- 8 drawn: no mirror
  return d[dir <= 4 and dir + 1 or 9 - dir][o.f], dir > 4
end
```

The game draws the frame with `sspr(..., flip)`. Each frame carries an anchor (the point between the feet), so a mirrored frame is placed with the anchor mirrored too: the game computes `x - (width - anchor_x)` for a flipped frame.

One module for all the animations

Yharnam plays the animations of the hunter and of every creature with one small module, `ANIM` (the module `anim` in `carts/yharnam/main.lua`). Copy it into a game that has actors with directions.

An actor is a table with `anim` (the name that plays), `f` (the frame, 1 is the first), `ft` (the ticks the frame has been on screen), `newf` (true in the tick a frame began) and `rev` (true when it plays backwards). A set holds, for each name, `t` (the ticks each frame lasts), `loop` and `d` (the frames of each direction).

Call	What it does
`ANIM.play(o, set, name, [rev])`	starts an animation; with `rev` it starts from the last frame and goes backwards
`ANIM.step(o, set, [rate], [carry])`	goes on by `rate` ticks (1 if missing); gives `true` when an animation that does not loop is over; with `carry` the ticks a frame did not need go to the next one
`ANIM.frame(o, set, dir)`	the frame to draw for the direction `dir` (0 S, 1 SE ... 7 SW) and `true` when it is to be drawn mirrored

COPY AND PASTE

```
ANIM.play(hunter, HUNT, "slash1")                  -- start a blow
if ANIM.step(hunter, HUNT, 1, true) then           -- once per tick
  ANIM.play(hunter, HUNT, "idle")                  -- the blow is over
end
local fr, flip = ANIM.frame(hunter, HUNT, hunter.dir)
sspr(fr[1], fr[2], fr[3], fr[4], x, y, flip)
```

The game reads `newf` and `f` to know when a blow lands (the frame number `hit` of the animation).

From the keys to the direction

bmlib gives the direction of a move: `lib.dir8(x, y)` is one of 8 numbers (0 right, 1 down-right, 2 down ... 7 up-right, `nil` for no move), and `lib.DIR8[d + 1]` is the unit vector of it.

How to make the 8 directions

- By hand: draw 5 and mirror 3. In bm Pixel, **h** flips a selection left-right and **y** draws mirrored; copy a direction, paste, flip.
- Pre-rendered: bm Animator's page F4 draws 1, 2, 4 or 8 directions of an animation in one go (see "Pre-rendered 2D").

BEWARE

[IN DEVELOPMENT] A command in bm Pixel that clones the sequences of SE, E, NE as SW, W, NW (a horizontal flip that keeps the hand that holds the weapon). Today the mirror is done by the game when it draws. Planned: beta rev 1.

Optimising sprites

Mirroring

Draw one side, mirror the other at draw time: `spr`, `sspr` and `zspr` all take `flip_x`. It halves the drawings of a side view and saves three directions of eight. Two cautions:

- a mirrored character changes hand: a sword in the right hand is now in the left. If it matters, draw the left side apart;
- the boxes and anchors must be mirrored too. bmlib's `H:zone` takes the flip for the boxes.

Reusing parts of sprites

Titan Clash draws the robot in layers over a base. The build script (`carts/titan/build/mkrobot.py`) keeps in its own layer everything that can change: armour plates, the near shoulder armour (intact or cracked), arm guns, the sword (on the back or in the hand). A layer holds only the pixels its parts show in front of the base, so base plus layers is exactly the robot with those parts.

The game draws a base and then its layers in a fixed order, with the same anchor:

COPY AND PASTE

```
-- piece(): {sx, sy, w, h, dx, dy} of the sheet, anchor at (x, y)
local function piece(r, x, y, flip)
  if flip then
    sspr(r[1], r[2], r[3], r[4], x - r[5] - r[3], y + r[6], true)
  else
    sspr(r[1], r[2], r[3], r[4], x + r[5], y + r[6])
  end
end
```

One frame and many combinations of equipment cost the frame once and each layer once.

Other ways of the same kind, from the build of *Yharnam* (`carts/yharnam/build/mkassets.py`): each frame is cut to its own box and packed on the sheet, and identical frames are kept only once.

Palettes

- The sheet has a palette of up to 256 colours; **F3** of bm Pixel edits it (add, remove, move, sort by hue, take the colours used by the sheet).
- A sheet with at most 256 colours is saved as `SHEET8`: a palette and runs, instead of 4 bytes a pixel. *Titan Clash* has a sheet of 2048 x 3376 pixels in 1.7 MB this way (`--sheet8`).
- Build the colours as ramps: *Titan Clash* gives each material 5 or 6 tones, from dark to light, and a glowing material (visor, vents) its own.
- The console shows 16 bits a colour (RGB565): bm Pixel shows next to each colour how it looks on the console.
- A second colour set for a second player (*Titan Clash* makes crimson armour for player 2) is made when the sheet is built. There is no function in the API that swaps a colour at run time. **[IN DEVELOPMENT]** a run-time palette swap: the API documents none.

Darkness and glow with the same palette

The light of *Yharnam* does not repaint sprites: each pixel has a light level and its colour goes through a *fade table* chosen by the game, so shadows turn blue and the colours near a lamp turn orange while staying on the sheet's colours. The calls are `fades`, `dark_begin`, `glow` and `dark_end`.

Sprite stacking

A sprite stack is a pseudo-3D object made of slices. You draw each slice as a flat picture. The slices lie side by side on the sheet. The game draws them one above the other and can turn the pile with an angle, so it looks like a solid object that turns. A tree, a barrel or a car can be a stack.

Call	What it does
`stack(sx, sy, w, h, n, x, y, [rz, scale, step])`	draws `n` slices of `w` by `h` pixels that lie side by side on the sheet from `(sx, sy)`, left to right. Slice 1 is the bottom and is drawn first. `(x, y)` is the centre of the bottom slice. Each slice turns by `rz` radians, is `scale` times bigger and sits `step * scale` pixels above the one below (`step` is 1 by default). Colour 0 is transparent
`stackv(sx, sy, w, h, n, x, y, [rz, scale, step])`	the same, but the slices lie one under the other on the sheet, top to bottom; the top one is slice 1
`zstack(name, x, y, [rz, scale])`	draws a stack zone by its name, with `step` 1
`zone(name)`	`x, y, w, h, frames, fps, slices` of a zone; `slices` is 0 for a plain zone and 2 to 255 for a stack zone

A **stack zone** is a line of the zones file: `stack name x y w h n` for `n` slices side by side, or `stackv name x y w h n` for slices one under the other, from `(x, y)` down. The bottom slice comes first. The zone has one frame.

- **bm Pixel, page F4** makes and changes `stack` zones, shows the slices one by one and shows the whole stack as it turns. A slice is drawn on F1. The chapter "F4: the stack" of the bm Pixel manual has the keys.
- **bm Animator** keeps the slices of a zone when you bake the zone again, if the new zone still fits the sheet.
- **From a stack to a mesh.** On the PC, `python tools/stack2mesh.py SHEET.png -o OUT.bmm --size W H -n N --at X Y` reads a stack and writes a `.bmm` file with one model. Each opaque pixel is a voxel, and voxels of one colour join into boxes. `--dir v` is for slices one under the other. The default limit is 4096 vertices; `--kernel-limits` allows 65535 for the RGB30. A stack that is too big stops with an error that gives the limit.

COPY AND PASTE

```lua
-- a tree of 8 slices of 16x16 pixels, turning
local a = 0
function _update() a = a + 0.02 end
function _draw()
  cls(1)
  zstack("tree", 80, 60, a, 2)
end
```

Caveats

- `stack` draws on the CPU, reads the sheet every call and keeps no memory. The speed on the Pi Zero W is **not verified on hardware**.
- A slice that is not wholly inside the sheet is skipped. A `scale` of 0 or less, or above 64, draws nothing.
- bm Pixel F4 does not delete a stack zone and does not make `stackv` zones: write those lines in the zones file.
- **[IN DEVELOPMENT]** A button in bm Pixel or bm Studio for the conversion of a stack into a mesh: today it is the command above. Planned: beta rev 1.

Pre-rendered 2D

What it does

Pre-rendering draws a 3D model, pose by pose and from each direction, into pixel art, once, before the game runs. The game then draws only 2D: it costs what a sprite costs, with the proportions and the light of a 3D figure.

How the games of the repository do it

- *Titan Clash*: `carts/titan/build/mkrobot.py` builds a robot of simple solids hung on a skeleton, renders each pose orthographically at 1:1 pixel size and stylises it: flat facets in a few tones per material, dark outlines on the silhouette and where a part stands in front of another, glowing visor and vents. It also gives the hitboxes and hurtboxes from the bones.
- *Yharnam*: `carts/yharnam/build/mkassets.py` renders over 5,000 frames (a few seconds each) of the hunter and the creatures, with signed distance fields, and the hunter's animations as 8-direction curves. Only the frames whose pose or model changed are drawn again.

These scripts run on the PC (they need numpy and Pillow); their output (the sheet and the list of frames) is in git, so building the game needs neither.

The same thing on the console: bm Animator F4

The **sprites** page of bm Animator draws a model's animation with the console's 3D engine into the sprite sheet, all on the console. Open a model that has a skeleton and at least one animation (see "3D models"), go to **F4**.

Row	Values
animation	one of the model's (or the rest pose)
frames	1 to 16
size	16, 24, 32, 48, 64, 96 or 128 pixels
directions	1, 2, 4 or 8
from above	0 to 75 degrees, in steps of 15
camera	flat or perspective
light	on, or off (flat colours)
outline	dark or none
colours	all, 32, 16 or 8
fill	50 % to 160 % of the frame
hitboxes	from bones, or none

[TO FILL: shots/editor-animator-sprites.png]

Enter draws the sprites and puts them in the sheet, below what it already has, one row of frames per direction: frame `f` of direction `d` is at `x0 + f * size`, `y0 + d * size`. The page adds a zone named `model_animation` in small letters and shows the `sspr()` call for the other directions.

The hitboxes from the bones

With **hitboxes** on, each bone becomes a rectangle in every frame. The name of the bone decides the kind: `hit`, `weapon`, `punch`, `fist`, `sword`, `blade`, `spear` or `attack` in it gives a hit box (red); `root`, `body`, `hips` or `pelvis` a body box (green); anything else a hurt box (blue).

Touch up with bm Pixel

The baked frames are ordinary sprites in the sheet. Open the project in bm Pixel, go to the zone (the panel says `zone "name" frame n/m`), and fix a pixel, add a highlight or flip a frame. The box preview (**j**) shows the boxes over your drawing. If you bake again, the zone with the same name is replaced, so touch up last.

Good to know

- The total of one bake must stay under 512 x 512 pixels (frames times size by directions times size). Otherwise: `too many, fewer frames or a smaller size`.
- `zspr` draws the first direction. For the others use `sspr()` with the numbers on the line at the bottom of F4.
- The light off and 16 or 8 colours give a hand-drawn look.

What F4 does not do yet

BEWARE

[IN DEVELOPMENT] Zones with a direction suffix. F4 makes one zone per animation, directions in rows; the design wants names like `hunter_walk_s`, `hunter_walk_se` and a ground anchor at the foot. Planned: beta rev 1.

[IN DEVELOPMENT] Socket points. The 2D position of a hand or a muzzle is not exported with the sprites (in 3D use `bone3d()`). Planned: beta rev 1.

[IN DEVELOPMENT] Layers. A bake is one block: no separate armour and weapon layers like *Titan Clash* (the design calls it a modular bake). Planned: beta rev 1.

[IN DEVELOPMENT] Event markers on the timeline (a hit frame, a sound): the game counts the frames itself. Planned: beta rev 1.

Try it, step by step

1. Open the **Dev** tab and start **bm Animator**. Choose a project with a rigged model (`carts/overbit` has rigged models).
2. Press **F4**. Choose the animation `walk` in the first row.
3. Set `frames` to 8, `size` to 48, `directions` to 4.
4. Check that `hitboxes` says `from bones`.
5. Press **Enter** and wait for `N sprites ... put in sheet`.
6. Press **Ctrl+S**.
7. Open the project in bm Pixel, press **F2**, look at the new rows, and fix what you like.

TRY IT

Rename a bone to `sword` in F2 of bm Animator, bake again and see its box turn red.

Meshes

What it does

A mesh is a 3D shape made of points (vertices) and triangles (faces). In bm there are three kinds, and moving among them is how a game's shapes become models you can edit.

Letter	What it is	In the game
M	a model of the MESH section (bm Studio's)	`model("name")`
C	a mesh in the code, written by bm Mesh as `mesh_name()` between `-- [bm Mesh begin]` and `-- [bm Mesh end]`	`local m = mesh_name()`
G	a mesh the code builds with `mesh()` , `mesh_sphere()` or `mesh_cube()`	the game's own code

From an image to a mesh

An image becomes a model in three ways.

- **In bm Studio, on the models page, `m` ** (or *Model from picture...* in the menu): a picture from the `pics/` folder of the SD or from the SD root (`.png` or `.jpg`). **Cutout** is a thick paper figure with the picture on the front and mirrored on the back; **lathe** turns the half outline around the vertical axis (vases, towers); **meshy.ai** is an online service that needs the network and an account.
- **On the PC, `tools/cutout2mesh.py` **: the same cutout and lathe with no network and no AI. Into a new cartridge the picture goes on the sheet; into an existing one the faces take flat colours.
- **On the PC, `tools/img2mesh.py` **: a vision model (it needs the Anthropic API and a key outside the repository) looks at the picture, writes a script of parts, and the script is built, drawn and corrected for a few rounds. The result is a `.bm` with a model, with bones and animations if the script has them.

COPY AND PASTE

```
python3 tools/cutout2mesh.py hero.png -o hero.bm --name hero
python3 tools/cutout2mesh.py vase.png -o vase.bm --lathe
```

From a mesh to an image

A model becomes an image in the ways of the chapter "Pre-rendered 2D" (bm Animator F4). On the PC, bm-forge draws previews:

COPY AND PASTE

```
python .claude/skills/bm-forge/scripts/bm_forge.py preview --help
```

[TO FILL: the options of `preview` for a single sprite-sized still; the bm-forge notes list `--turntable` , `--clip` , `--mask` and `--compare` , and say a render is a PNG or a GIF]

From a mesh to a model and back, with bm Mesh

On the **list** page of bm Mesh:

Key	What it does
m	copy as a model (mesh to model): a G or C mesh becomes an M model you can name and save
c	copy as code (model to `mesh_name()`): the model becomes a C mesh
r	rename
d	duplicate (a model keeps its skeleton)
Enter	edit it

Editing a G mesh makes a copy of it as a model first: the game's code is not rewritten. A copy as code does not keep the skeleton.

Editing vertices and faces

On the **edit** page (F2) **Tab** switches between vertices and faces.

Key	What it does
g, r, t	move, rotate, scale the choice
x	extrude the chosen faces
m, M	mirror the choice / copy it to the other side of 0
j	make a face on 3 or 4 chosen vertices
k, K	merge the vertices / weld those at the same point
u	split each chosen face into 4
i	turn the faces over
p, o, c	paint, take a colour, palette

Use **M** to build a symmetrical character: model half, copy it to the other side.

Where the game's meshes come from

`cart_meshes(path)` runs the game's code apart, in a Lua state of its own, once through `_init`, `_update` and `_draw`, and collects what `mesh()`, `mesh_sphere()` and `mesh_cube()` build; the name of a mesh is the name of the variable that holds it. A mesh built only after some input may not be found.

Caveats

- At most 65535 vertices and 65535 triangles in a mesh, and 256 models in a cartridge.
- bm Mesh keeps the skeleton of a model: the bone of each new vertex is the one it comes from.
- A sprite stack becomes a mesh with `tools/stack2mesh.py` (chapter "Sprite stacking"). A button for it in bm Studio or bm Mesh is not there yet.

3D models

What it does

For a character that is 3D in the game, the chain is: **model** (bm Studio) → **skeleton** and **animations** (bm Animator) → the game draws it with `model()`, `animate()` and `draw3d()`.

bm Studio: the model

bm Studio builds models out of blocks and tiles, on the console.

- Tools: **1** block (a cube), **2** tile (one side of a cell: floors, walls, doors), **3** select (move, turn, flip, mirror, copy faces), **4** vertex (move corners: roofs, ramps, free shapes), **5** paint (pixels of a face's tile).
- The textures are pieces of the project's sprite sheet. There is no colour at the corners: a face is either a tile or one flat colour.
- **F2** is the models page: new, rename (up to 16 letters), duplicate, delete, reduce triangles (`-`), model from a picture (`m`).
- The console draws about 1200 triangles in a scene at 60 fps; bm Studio warns above that with `heavy for 60 fps`. **-** reduces a model, keeping the borders, the colour lines and the texture seams; the skeleton follows.
- A model has at most 4096 vertices and 16384 triangles when saved from bm Studio.

bm Animator: the skeleton and the animations

- **F2 rig**: **n** makes the skeleton (the bone `root`) and adds bones; **m** mirrors a bone and its children, with names ending in `.L` and `.R`; **k** gives each face to its nearest bone (rigid, good for robots), **K** each corner (smooth); **v** chooses faces by hand. At most 64 bones.
- **F3 animate**: keyframes on a 12 frames per second timeline; every turn of a bone is a keyframe; **i** chooses linear, smooth or step; **m** mirrors a pose, so a walk needs two poses and a loop. At most 255 animations a model.
- **F1 play**: check the animation, blend it with the next one (`b`), show the skeleton (`k`).
- Skinning is rigid: each corner follows one bone, no mixed weights.

Name the bones with the game in mind

The names have a use: `.L` and `.R` for the two sides; `sword`, `fist` and the like for the boxes of an attack; `root`, `body`, `hips` for the body; `head` for a headshot with `hit3d`.

Models outside the console

- **bm-forge** (`.claude/skills/bm-forge`): builds MESH models from box JSON or untextured `.glb`, skeletons and clips from JSON, and exports `.bm` (Pi) and `.b16` (RGB30). A person or character is one continuous skin with shared vertices at the joints, rigged, never loose boxes. The template `examples/charkit/` has two presets.
- A local `.glb` from the SD card does not open in the editors yet; importing it in the suite is planned (APP-9). Today the console gets a `.glb` only from the picture-to-3D service (`picture3d`), and bm Studio then takes it as a model. On the PC, `python3 scripts/bmres.py convert IN.glb OUT.bmm` converts a `.glb` to a `.bmm`; `tools/meshy2mesh.py`, `tools/local2mesh.py` and `tools/cutout2mesh.py` make models from other sources. For a game of the repository, the build reads `assets/models.bm` or `assets/models.glb` (`models.bm` first) and puts the

models in the `.bm`. `carts/village/assets` holds `models.bm`, not a `.glb`.

- **[IN DEVELOPMENT]** Saving and opening `.bmm` model files from the console's bm Studio. Today the program saves only the project; on the PC `scripts/bmres.py extract GAME.bm --model villager -o VILLAGER.bmm` makes one.

Caveats

- A model has no skeleton until bm Animator makes one; `animate()` gives an error if the model has none.
- **[TO FILL: the size of a model that the Pi Zero W draws smoothly; the notes give the 1200 triangles a scene as the figure for 60 frames per second]**

Drawing and animating in the game

All these calls are in the 3D API (`docs/api/light3d.md`).

The model and its animation

Call	What it does
`models()`	the names of the cartridge's models
`model(name)`	the model as a mesh, with the sheet's texture; do it in `_init`
`clips(m)`	the animations of a model: `{ {name=, length=, loop=}, ... }`
`animate(m, [anim, t, anim2, t2, k, bone])`	puts the model in the pose of `anim` at time `t` in seconds; with `anim2, t2, k` it blends two animations; with `bone` the second holds only for that bone and those under it (a torso that shoots on running legs)
`bone_turn(m, bone, [rx, ry, rz])`	turns a bone on top of the animation: aiming
`bones3d(m)`	the names of the bones
`bone3d(m, bone)`	where the head and the tail of a bone are in the last pose: hands, a sword's tip, a light
`hit3d(m, x, y, z, ry, scale, ox, oy, oz, dx, dy, dz, [maxd])`	a ray against the bones in the last pose
`bounds3d(m)`	the box round the mesh
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	draws the posed model

`animate` costs only the bones. Call it before `draw3d` , every frame.

Light, shadow, shading

Call	What it does
`camera3d(x, y, z, [yaw, pitch, fov, roll])`	the camera
`light3d(x, y, z, [ambient])`	the direction of the light and the ambient light
`sky3d(sun, sky, ground)`	the sun's colour and the ambient light from above and below
`shine3d(spec, exponent, rim)`	reflections on glossy faces and light on the rim
`lamp3d(i, x, y, z, radius, [k, colour])`	one of 4 point lights
`fog3d(colour, near, far)`	fog
`shadow3d(style)`	the shadows of `draw3d` with flag 8: 0 darken, 1 dithered black

Call	What it does
`zclear()`	clears the z-buffer, every frame before `draw3d`
`cel3d([bands, floor, outline_rgb, outline_width])`	cel light for `draw3d` calls with flag 256 (2 to 8 bands, off by default) and the colour and width of the outline for flag 512; with no argument it returns the values

The flags of `draw3d`

Flag	Effect
1	no z-buffer
2	no light (flat colours)
4	smooth (Gouraud)
8	the shadow of the mesh on the plane `y`, along the sun (the mesh is not drawn)
16, 32, 48	level of detail 2, 1, 0
64	in front (first-person arms and weapons)
128	flat textures (the average texel)
256	cel light: the sun light of this draw is cut into bands (see `cel3d`)
512	outline of the mesh, after it (see `cel3d`)

Materials in a face's colour

A face's colour can carry **material bits**: bit 30 `0x40000000` emissive (visors, lights), bit 29 `0x20000000` glossy, bit 28 `0x10000000` screen door (shields, glass), bit 27 `0x08000000` flat, bits 24 to 26 the level of detail.

Shaders

The V3D driver of the Pi runs QPU shaders written in `tools/qpuasm.py` and generated into `src/gpu/shaders.h`; the Mali of the RGB30 has its own bring-up (`docs/progress/graphics-mali.md`). A game does not write them: it chooses with `gpu3d()` (on, 4x anti-aliasing, where the vertex shader places the vertices) and the Settings rows. The same functions draw on the ARM when the GPU is off.

> BEWARE
> **[IN DEVELOPMENT]** Shaders written by the game. The API documents no call to give a model a shader of its own.

Code to copy

COPY AND PASTE

```
local v, t = nil, 0
function _init()
  v = model("villager")
end

function _update()
  t = t + 1 / 60
end

function _draw()
  cls(1)
  zclear()
  camera3d(0, 1, -4)
  light3d(-0.4, 0.8, -0.5, 0.4)
  animate(v, "walk", t)
  draw3d(v, 0, 0, 0, 0, 0, 0, 1)
  draw3d(v, 0, 0, 0, 0, 0, 0, 1, 8)
end
```

The last line draws the shadow. This is the pattern of the games with rigged models, such as `carts/overbit` .

Tutorials

A side-scroller hero

1. In bm Pixel press **z** until the size is 16x16. Draw the hero facing right in the first sprite.
2. Press **PgDn** and draw the second frame of the run, then the third and the fourth, to the right of each other.
3. Press **+** until the panel says 4 frames and **>** or **<** for the speed. **p** plays.
4. Save with **Ctrl+S**.
5. Write the zone in `zones.txt` (`hero_run 16 32 16 16 4 10`) and build with `mkbm.py --sprites zones.txt` .
6. In the game: `zspr("hero_run", x, y, nil, facing_left)` .

A top-down hero with 4 directions

1. Draw 2x2-cell sprites (size 16): facing down (two walking frames), facing up (two), facing right (two).
2. Put them 4 cells apart on the sheet, as in the code of "Top-down characters, 4 directions".
3. Draw with `spr(s, x - 8, y - 14, 2, 2, flip)` and `flip` true when walking left.

A 3D character baked into 8 directions

1. Build the model in bm Studio (tools 1 to 5), at most 1200 triangles.
2. In bm Animator F2 make the skeleton and name the weapon bone `sword` ; press **k**.
3. In F3 make `idle` and `walk` (two poses and a loop).
4. In F4 set `frames` 8, `size` 48, `directions` 8, `hitboxes` from bones; press **Enter**.
5. Open the project in bm Pixel and touch up the zone.
6. In the game: `zspr("hero_walk", x, y)` for the first direction and `sspr()` for the rest, with the numbers shown in F4. Mirror the three left directions as in "Top-down characters, 8 directions".

Tips and limits

Limits

What	Limit
sprite sizes in bm Pixel	8, 16, 32, 64, 128 pixels
frames of a zone in bm Pixel	1 to 16, speed 1 to 30 fps
palette	1 to 256 colours
sprite sheet	4096 x 4096 pixels
one bake (F4)	512 x 512 pixels in all, sprites 16 to 128 pixels, 1 to 16 frames
bones in a skeleton	64
animations of a model	255
triangles a scene at 60 fps	about 1200
point lights (`lamp3d`)	4
models in a cartridge	256

Tips

- Pick the look first: hand-drawn, pre-rendered or 3D. Changing later means drawing again.
- Mirror what you can, and keep the anchor of each frame at the feet.
- Keep the frames of an animation next to each other, on one row.
- Name bones for the boxes they will make.
- Bake last, touch up after the bake, and keep a copy of the project before baking again.
- Try the game often with **F5**: it saves and comes back to the same page.

When something goes wrong

- `too many, fewer frames or a smaller size` : the bake passes 512 x 512 pixels; lower the frames, the size or the directions.
- `NO SKELETON` in bm Animator: make one in F2.
- `animate` gives an error: the model has no skeleton or the animation name is wrong; check with `clips(m)` .
- The mirrored sprite is off to one side: the anchor was not mirrored with the frame.

Open points

- **Pre-rendered 2D**: picture to take: shots/editor-animator-sprites.png
- **Meshes**: the options of `preview` for a single sprite-sized still; the bm-forge notes list `--turntable`, `--clip`, `--mask` and `--compare`, and say a render is a PNG or a GIF
- **3D models**: the size of a model that the Pi Zero W draws smoothly; the notes give the 1200 triangles a scene as the figure for 60 frames per second